B2B Lead & Account Customer Lifetime Value (CLV) Modelling on Salesforce + HubSpot + Snowflake — An Implementation Guide for Data Scientists and Data Engineers

TL;DR

- Build a **two-stage GBDT** as the production workhorse — a LightGBM/XGBoost/CatBoost classifier for P(convert) multiplied by a Tweedie or Zero-Inflated Lognormal (ZILN) regressor for E[value|convert] — because tree ensembles are the empirically established state of the art on tabular CRM data at B2B sample sizes (thousands of accounts, not millions), while classic B2C "buy-till-you-die" (BTYD) models like BG/NBD are used only as a benchmark and a feature source, not the primary engine.
- The single biggest determinant of success is **not the algorithm but point-in-time-correct feature engineering off CRM history objects** (Salesforce `OpportunityFieldHistory`/`OpportunityHistory`/`LeadHistory` and HubSpot property-history endpoints), assembled with Snowflake `ASOF JOIN` and the Snowflake Feature Store's `generate_training_set(spine_timestamp_col=...)` so no feature ever "sees the future"; mutable CRM fields (`Lead.Status`, `Opportunity.StageName`/`Probability`/`ForecastCategory`, `hubspotscore`, `lifecyclestage`) are the leakage traps that silently inflate offline metrics.
- Everything is implementable natively on the stack: Fivetran/native connectors → Snowflake (bronze/silver/gold via Dynamic Tables or dbt) → Snowflake Feature Store + Model Registry → batch scoring → Reverse ETL write-back to a custom `pCLV__c` field on Salesforce Lead/Account and a custom HubSpot property; monitor with Snowflake ML Observability (`CREATE MODEL MONITOR`, which supports PSI drift). Field-name and edition-gating caveats are flagged throughout — verify custom fields and Einstein/Marketing-Hub-tier features in your own org.

Key Findings

1. **B2B is a hybrid contractual/non-contractual setting**, so no single BTYD model fits an account that spans install-base refresh (contractual/discrete) and peripheral reorders (non-contractual/continuous). A covariate-driven ML engine is the right primary tool.
2. **CLV is zero-inflated and heavy-tailed**, mandating a two-part hurdle formulation: `Value of a lead = P(convert) × E[value | convert]`.
3. **History objects are mandatory**, not optional: they are the only way to build point-in-time-correct features and correct labels. Current CRM state is future information relative to the scoring moment.
4. **GBDTs win on tabular CRM data at B2B scale** (empirical literature below); deep sequence models only overtake them with very large, granular event streams.
5. **The stack is fully sufficient.** Snowflake's Feature Store, Model Registry, Dynamic Tables/Streams/Tasks, ASOF JOIN, and ML Observability cover the whole MLOps loop; Salesforce/HubSpot custom fields and Reverse ETL cover activation.
6. **Ranking ≠ causal allocation.** Prioritise the sales queue by pCLV, but allocate incremental

marketing budget by uplift models, or you will over-serve "sure things."

Details

1. Problem Framing and CLV Definitions for B2B

1.1 Formal definition

CLV is the present value of future net cash flows attributable to a customer relationship. The discounted-cash-flow (DCF) form is:

```
CLV = Σ_t [ m_t · s_t / (1 + d)^t ]
```

where `m_t` is expected margin in period `t`, `s_t` is survival/retention probability to `t`, and `d` is the periodic discount rate. Two stances: **Historical CLV** (sum of realised past margin) and **Predictive CLV (pCLV)** (expected future value over a horizon — what we model).

Margin vs revenue: model **margin-based** CLV where gross-margin data exists (a distributor deal at 12% margin is not equivalent to a direct enterprise deal at 45%). If margin is unavailable at lead time, model revenue and apply a segment-level margin multiplier downstream, documenting the approximation.

1.2 Fader & Hardie's 2×2 taxonomy and where B2B hardware sits

Fader & Hardie classify settings on **contractual vs non-contractual** (is churn observed or latent?) and **discrete vs continuous** (fixed epochs or any time?). B2B hardware + channel is a **hybrid**, the central modelling nuisance:

- The **direct install-base/refresh business** (Assets, warranties, service contracts) is **contractual-ish and discrete** — renewals/refreshes on semi-predictable cadences (BG/BB, survival fit here).
- **Transactional peripheral reorders via channel/e-commerce** are **non-contractual and continuous** — no "cancel" event; the customer just stops (Pareto/NBD, BG/NBD territory).
- One account spans both, so no single BTYD model covers it — a primary reason a **covariate-driven ML approach** (ingesting both transactional and contractual signals as features) is the preferred production engine.

1.3 Why B2B CLV differs from B2C (and why classic BTYD needs adaptation)

Classic BTYD (Pareto/NBD, BG/NBD, Gamma-Gamma) works on **transaction logs** reduced to recency, frequency, and average monetary value, assuming a large population of exchangeable customers with stationary latent purchase/death rates. B2B violates this:

1. **Small-N, large-value, heavy-tailed accounts** — thousands not millions; a few whales distort the fit and make population BTYD estimation noisy.
2. **Buying centres** — the "customer" is an org with a buying group (economic buyer, champion, IT admin, procurement); per-customer transaction logs throw this away.
3. **Long multi-touch sales cycles** — value decided over months of opportunities.

4. **Contract/renewal & refresh cycles** — predictable replacement timing BTYD's memoryless assumptions cannot express.
5. **Channel/distributor obfuscation** — when a reseller transacts, end-customer identity and reorder behaviour are partly invisible.
6. **Account hierarchies & land-and-expand** — value expands across parent-child hierarchies; a flat customer ID misrepresents it.
7. **Heavy right-censoring & non-stationarity** — most accounts still "alive"; portfolios/pricing/GTM shift over time.

Consequence: **use BTYD as a benchmark and feature generator** (its P(alive)/expected-transactions are excellent features), but drive production predictions with covariate-rich ML.

1.4 The business decisions the model must support (the decision defines the target)

Decision	Modelling target implied
Lead prioritisation & routing	Rank by **value of a lead** = P(convert) × E[12-mo value]
SDR/AE capacity allocation	Expected value per lead × conversion probability, calibrated
Marketing spend & CAC:LTV by channel/campaign	pCLV by acquisition source vs CAC per source
ICP definition	Which firmographic/behavioural features drive high pCLV
Partner/channel investment	Account-level pCLV aggregated by partner
Pricing/discount approval	E[value] and margin sensitivity; expansion probability
ABM targeting	Account-level pCLV over a fixed horizon
Renewal/expansion focus	Renewal probability & time-to-churn (survival)

Principle: **the decision determines whether you need a ranking, a calibrated probability, a dollar point-estimate, a timing estimate, or an uncertainty band** — don't build one number and hope it serves all.

1.5 Defining the prediction target(s) precisely

For a lead (pre-customer, cold-start), the standard **two-part / hurdle** decomposition is:

```
Value of a lead = P(convert) × E[value | convert]
```

This is standard because CLV is **zero-inflated and heavy-tailed**: most leads never convert (a spike at zero), and converters' value is right-skewed (log-normal-ish). Modelling the two parts separately (or jointly with a zero-inflated loss) beats forcing one regressor to straddle the zero-

mass and the tail.

Targets to define with the business, each over an explicit **horizon** (e.g., 12/24/36 months from lead creation): `p_convert`; `first_order_value`; `pCLV_H` (cumulative margin over H); `n_repeat` (BTYD-style); `renewal_prob`/`time_to_churn` (survival, for install-base accounts).

---

2. Data Foundation and Source-System Mapping (Salesforce + HubSpot + Snowflake)

2.1 How the data lands in Snowflake

- **Third-party ELT**: Fivetran, Airbyte, Stitch — managed SF & HubSpot connectors handling schema drift, incremental sync, soft deletes (the most common production choice).
- **Salesforce → Snowflake**: Data Cloud sharing; Bulk API 2.0 for large extracts; connectors use REST/Bulk APIs.
- **HubSpot → Snowflake**: CRM v3 API (objects, associations, **property-history** endpoints); incremental sync via `hs_lastmodifieddate`; search/list APIs are rate-limited.
- **Snowflake native**: connectors/Marketplace — verify current native-connector availability per source in your account.

Critical for CLV: capture **change history**, not just current state. Incremental upserts of the latest row destroy the point-in-time signal. Either ingest the CRM **history objects** explicitly, or build **SCD Type 2** tables from the change stream. Handle **soft deletes** (`IsDeleted`, `archived`) as first-class — a deleted opportunity is information.

2.2 Salesforce object & field inventory

Standard objects/fields below are from the Salesforce Object Reference; anything `__c` is org-specific — confirm in your instance. Field-history tracking is enabled **per field** (max 20/object) with limited retention unless Field Audit Trail is licensed — **verify what your org tracks**.

Lead: `Status`, `LeadSource`, `Rating`, `Industry`, `NumberOfEmployees`, `AnnualRevenue`, `Country`/`State`, `Title`, `CreatedDate`; conversion linkage `IsConverted`, `ConvertedDate`, `ConvertedAccountId`, `ConvertedOpportunityId`, `ConvertedContactId`. ⚠️ **Leakage**: `Status`, `Rating`, `IsConverted`, `ConvertedDate` update *after* the outcome — never use post-creation values for a creation-time prediction.

Account: `Type`, `Industry`, `AnnualRevenue`, `NumberOfEmployees`, `ParentId`, `OwnerId`, `BillingCountry`/`BillingState`, `Sic`, `NaicsCode`, `Website`.

Contact: `AccountId`, `Title`, `Department`, `Email`, `LeadSource`, `CreatedDate`.

Opportunity: `Amount`, `StageName`, `Probability`, `CloseDate`, `IsWon`, `IsClosed`, `ForecastCategory`, `Type`, `LeadSource`, `ExpectedRevenue`, `HasOpportunityLineItem`, `AccountId`, `OwnerId`, `CreatedDate`. ⚠️ **Leakage**: `StageName`, `Probability`, `ForecastCategory`, `IsWon`, `IsClosed`, final `Amount`, `CloseDate`.

Line-item/product: `OpportunityLineItem` (`Quantity`, `UnitPrice`, `TotalPrice`, `Product2Id`, `PricebookEntryId`), `Product2` (`Name`, `Family`, `ProductCode`), `PricebookEntry`. **Relationship**:

`OpportunityContactRole` (`ContactId`, `Role`, `IsPrimary`) — the buying-group signal. **Marketing:** `Campaign`, `CampaignMember` (`Status`, `HasResponded`, `FirstRespondedDate`). **Activity:** `Task`/`Event` (`Type`, `ActivityDate`, `WhoId`, `WhatId`, `CreatedDate`). **Service:** `Case` (`Priority`, `Status`, `CreatedDate`, `ClosedDate`, `Type`), `Entitlement`. **Commerce/Contract:** `Quote`/`QuoteLineItem`, `Order`/`OrderItem`, `Contract` (`StartDate`, `EndDate`, `Status`).

Asset (install base — central for hardware): `AccountId`, `ContactId`, `Product2Id`, `SerialNumber`, `InstallDate`, `PurchaseDate`, `UsageEndDate`, `LifecycleStartDate`, `LifecycleEndDate`, `Status`, `Price`, `Quantity`, `ParentId`, `RootAssetId`, `CurrentMrr`, `CurrentAmount`, `CurrentQuantity`. These give device counts, warranty expiry, and refresh timing — the strongest repurchase predictors for a hardware vendor.

History/audit objects (ESSENTIAL): `OpportunityFieldHistory` ("Represents the history of changes to the values in the fields of an opportunity"); `OpportunityHistory` (special object tracking Amount/Probability/Stage/CloseDate stage progression — stage history is **not** auto-deleted); `LeadHistory`, `AccountHistory`; Field Audit Trail. They are the only way to reconstruct record state at prediction time and to build labels — without them your model will leak. Salesforce Developers

2.3 HubSpot object & property inventory

Verified against HubSpot CRM API / default-property docs. Many analytics properties are **auto-set/read-only**; several are **edition-gated** (flagged).

Contacts — analytics/engagement (mostly Marketing-Hub-populated, auto-set): `hs_analytics_source` ("The first known source through which a contact found your website"), `hs_analytics_source_data_1`, `hs_analytics_source_data_2` (read-only drill-downs), `hs_latest_source`; `hs_analytics_num_page_views`, `hs_analytics_num_visits`, `hs_analytics_num_event_completions`; `hs_analytics_first_touch_converting_campaign`, `hs_analytics_last_touch_converting_campaign`; `hs_email_open`, `hs_email_click`, `hs_email_first_open_date`, `hs_email_last_send_date`; `num_conversion_events`, `recent_conversion_event_name`, `notes_last_contacted`, `num_notes`. Scoring/stage: `hubspotscore` (rule-based), `hs_lead_status`, `lifecyclestage`, and predictive `hs_predictivecontactscore` (label "Predictive Lead Score") and `hs_predictivecontactscorebucket` (label "Lead Rating"). ⚠️ **Flag:** the exact internal name is `hs_predictivecontactscore` (a historic `..._v2` name existed); confirm in your portal via the properties API. Predictive scoring is **edition-gated** (Marketing Hub Enterprise). All are ⚠️ **leakage-prone** (post-hoc updated) and edition-dependent.

Companies: `numberofemployees`, `annualrevenue`, `industry`, `hs_num_child_companies`, `domain`, `country`, `hs_analytics_source`.

Deals: `amount`, `dealstage`, `hs_deal_stage_probability`, `closedate`, `days_to_close`, `hs_acv`, `hs_arr`, `hs_mrr`, `hs_tcv`, `hs_margin*`, `dealtype`, `num_associated_contacts`, `hs_is_closed`, `hs_is_closed_won`, and stage-timing calculated properties (Date entered/exited stage; "Time in current stage"). ⚠️ Recurring-revenue metrics (`hs_arr`/`hs_mrr`) require **Sales Hub Enterprise**. ⚠️ `dealstage`, `hs_deal_stage_probability`, `amount`, `closedate` are leakage-prone.

Other objects: Tickets, Line Items & Products, Marketing Emails/Email Events, Forms & Form

Submissions, Web Analytics, Lists, Workflows, Marketing Events, engagements (Calls/Meetings/Notes). The **Associations API** (with labels) connects contacts↔companies↔deals — essential for account rollups and buying-group features.

Property history: current-value endpoints give only the latest value. Point-in-time-correct HubSpot features require the **property-history** capability (v3 `propertiesWithHistory` / history endpoints) — otherwise `lifecyclestage`, `hubspotscore`, `dealstage` leak.

2.4 Identity resolution across Salesforce & HubSpot (and the channel gap)

- **Join key**: company **email domain** is the most reliable B2B join key. Normalise domains (strip `www`, lowercase, handle `.co.uk`).
- **Lead-to-account matching (L2A)**: match by domain, then fuzzy company-name (Jaro-Winkler/edit distance), then firmographic tie-break — implementable in Snowflake.
- **Account hierarchy rollup**: recurse `Account.ParentId` to the ultimate parent (recursive CTE below).
- **Channel/reseller obfuscation**: when the transacting account is a distributor, the end customer is often a `ShipTo`, an Asset `AccountId`, or an Order field — not the billing account. Build an **end-customer resolution layer** preferring Asset/registration identity, and explicitly flag unknown-end-customer records (they need the cold-start path). Never silently attribute distributor volume to the distributor "account" — it destroys per-account CLV.

2.5 Third-party firmographic & intent enrichment

Common: firmographics (employees, revenue, NAICS/SIC, age, public/private), technographics, intent data, website engagement. Realism: enrichment is **costly and imperfectly populated**, with better coverage for large firms than SMBs, biasing models. In the B2B lead-scoring literature firmographic cues support **early-stage segmentation** but behaviour dominates once available. The two-stage PRISM model (Andreev et al., "Profiling before scoring: a two-stage predictive model for B2B lead prioritization," *Journal of Personal Selling & Sales Management*, 2026, DOI 10.1080/08853134.2026.2633231) demonstrates the value of firmographics for cold-start: in commercial cleaning services PRISM reached a conversion rate of 85.65%, "surpassing the baseline model (67.62%) by 18.03% and 'classification only' (74.55%) by 11.1%," with gains reported across three B2B service industries. Treat enrichment as **most valuable at cold start** and measure incremental lift before paying.

---

3. Feature Engineering — The Most Important Section

For each feature: **source → exact field → transformation → rationale → expected direction → leakage risk.** Compute every windowed/aggregate feature **as of the prediction timestamp** (Section 3.3).

3.1 Feature catalog

(a) Firmographic

Feature	Source · field	Transform	Rationale / direction	Leakage risk
Employee count	SF `Account.NumberOfEmployees` / HS `numberofemployees`	`log1p`, bucketed	Proxy for seat/ device demand; ↑	Low (snapshot it)
Annual revenue	SF `AnnualRevenue` / HS `annualrevenue`	`log1p`; **missing flag**	Ability to spend; ↑	Missing-not-at-random
Industry	SF `Industry`/`NaicsCode`/`Sic`	target/ CatBoost enc.	Vertical device intensity	Low
Geography	SF `BillingCountry` / HS `country`	grouping	Coverage, channel, pricing	Low
Company age	founded/`YearStarted`	years	Stability; ambiguous	Low
Public/ private	enrichment	boolean	Budget cycle	Low
Hierarchy size	SF `Account.ParentId` recursion	# children, depth	Multi-site expansion; ↑	Low
Seat/TAM proxy	employees × industry device ratio	derived	Direct device demand; ↑	Low

(b) RFM and RFM-extensions (account level) — from **Opportunity/Order/Asset: Recency** (days since last won deal/order/asset install), **Frequency** (count of prior won deals/orders in window), **Monetary** (avg & total prior margin). **Engagement-RFM** (recency/frequency of activities, email opens, web sessions) is often more predictive than transactional RFM for **leads** who have no transactions yet. Fader & Hardie showed R and F are the **sufficient statistics** for future transaction count in BTYD, and (Fader, Hardie & Lee 2005b, "RFM and CLV: Using Iso-Value Curves for Customer Base Analysis," *Journal of Marketing Research* 42(4):415–430) that monetary value is, to first order, **independent of frequency** — so model spend separately. This justifies the two-part structure and marks R and F as highest-value transactional features.

(c) Engagement / behavioural / digital — Web: `hs_analytics_num_visits`, `hs_analytics_num_page_views`; high-intent page visits (pricing/product/ROI) as separate counters. Content/events: form submissions (`num_conversion_events`, `recent_conversion_event_name`), webinar/marketing-event attendance, downloads. Email: `hs_email_open`, `hs_email_click` → rates and **recency**. Product telemetry/device registration if available. **Windowed aggregates** at 7/30/90/180/365 days; **time-decayed** sums; **velocity** (Δ between windows) and **acceleration**. **Engagement breadth (distinctively B2B)**: # distinct contacts engaged, # departments engaged — the buying-centre signal; ↑ breadth strongly ↑ value.

(d) Sales process / opportunity — # opportunities; **time-in-stage** & stage-progression velocity (from `OpportunityFieldHistory`); # **stage regressions**; discount level (list vs `UnitPrice`); **product-mix breadth** (# distinct `Product2.Family`); deal-size distribution; historical win rate; sales-cycle length; touch counts by type (`Task`/`Event`); meeting-to-opportunity ratio; owner/rep effect; **channel-partner identity**. ⚠️ Most are valid only **after** an opportunity exists — for pure lead scoring at creation, use account-history versions, not the current opp.

(e) Buying group / relationship graph — contacts per account; **seniority mix** via `Contact.Title` parsing (C-level/VP/Director/IC); # departments; champion presence; `OpportunityContactRole` role coverage (economic buyer + champion?); simple graph features (degree, # connected contacts). Richer buying groups ↑ value.

(f) Support / service — `Case` volume, severity mix, time-to-resolution, reopen rate, CSAT (if captured), RMA/warranty claims. High **unresolved** case load ↓ renewal; excellent service ↑ CLV.

(g) Install base / product (hardware-specific, high value) — from `Asset`: device count, distinct families owned, **warranty/`UsageEndDate` proximity**, `InstallDate`-derived age, refresh-cycle timing (age vs typical refresh interval per family), accessory/service **attach rate**. Engineer `months_to_expected_refresh` per family — an account whose fleet nears `UsageEndDate` is a high-probability refresh.

(h) Temporal / contextual — account tenure; fiscal-quarter/seasonality flags; cohort (acquisition quarter); **time since last purchase relative to expected refresh cycle**; macro indicators. Snapshot as-of.

(i) Marketing attribution / acquisition — `LeadSource`, `hs_analytics_source`, first/last-touch converting campaign, channel, CAC by channel. ⚠️ **Causal caution**: acquisition-channel features are predictive but **not causal** — use them for prediction/ranking, but for **spend allocation** move to uplift (Section 4.7), not raw pCLV-by-channel.

3.2 Feature selection methodology (top emphasis)

Families: **Filter** (variance threshold; Spearman correlation with target; mutual information; chi-square) — coarse first screen; **Wrapper** (RFE, forward/backward) — powerful but overfits outside a CV loop; **Embedded** (L1/Lasso; tree gain/split importance) — cheap and effective; **Permutation importance** — model-agnostic but **biased with correlated features**; **SHAP-based** (Lundberg & Lee 2017) — consistent attributions, best with correlated features; **Boruta** — all-relevant selection via shadow features; **Stability selection** — subsample+Lasso repeatedly, robust to small-N instability (very relevant for B2B).

Pitfalls: **multicollinearity** (revenue/employees/TAM correlated) — cluster and keep one representative or use SHAP/stability; **high-cardinality bias** — impurity/gain importance inflated, prefer permutation/SHAP on holdout or CatBoost encoding; **target leakage** (#1 killer, see 3.3); **selection outside the CV loop → optimistic bias** — do selection **inside** each fold (nested).

Recommended protocol: (1) enforce point-in-time correctness first — no leaky features enter the pool; (2) coarse filter (drop near-zero variance and high-missing unless missingness modelled); (3) cluster correlated features, keep one representative; (4) fit GBDT, rank by SHAP on a **temporal**

holdout; (5) confirm with stability selection across bootstraps; (6) keep the union of clear-signal features, documenting rationale and direction.

High-cardinality categoricals: prefer **CatBoost ordered target statistics** (Prokhorenkova et al. 2018), which specifically prevent the target-leakage/prediction-shift of naïve target encoding; alternatives are K-fold target encoding, hashing, or embeddings (with enough data). **Missing data**: CRM data is **missing-not-at-random** — a blank `AnnualRevenue` often signals a small/unqualified account, so **add explicit missingness indicator flags** and let LightGBM/XGBoost/CatBoost handle NaN natively (they route missing to the optimal split) rather than mean-imputing the signal away.

3.3 Point-in-time correctness and leakage (critical)

Core rule: every feature for a prediction at time `T` must use only data with effective timestamp `≤ T`. CRM records are **mutable**: the *current* value of `Opportunity.StageName` or `hubspotscore` reflects the future relative to the scoring moment. Using current state as a historical feature is the most common and damaging CRM-ML error.

Leaky fields to quarantine (never use post-`T` state): `Lead.Status`, `Lead.Rating`, `Lead.IsConverted`, `Opportunity.StageName`, `Opportunity.Probability`, `Opportunity.ForecastCategory`, `Opportunity.IsWon/IsClosed`, final `Amount`, `CloseDate`, HubSpot `lifecyclestage`, `hubspotscore`, `hs_predictivecontactscore`, `dealstage`, `hs_deal_stage_probability`.

How to build as-of features: reconstruct historical state from history objects (`OpportunityFieldHistory`, `LeadHistory`, `AccountHistory`, HubSpot property history), model each record as **SCD Type 2** (valid_from/valid_to), then `ASOF JOIN` the feature tables to a **spine** of (entity_id, prediction_timestamp). Snowflake's native `ASOF JOIN` "combines rows from two tables based on timestamp values" and finds, for each spine row, the closest earlier feature row via `MATCH_CONDITION`.

3.4 Runnable-style Snowflake SQL patterns

(1) Account-level RFM (as-of):

```sql
CREATE OR REPLACE DYNAMIC TABLE gold.account_rfm
  TARGET_LAG = '1 hour' WAREHOUSE = transform_wh AS
WITH won AS (
  SELECT AccountId, CloseDate, Amount FROM silver.opportunity WHERE IsWon = TRUE
)
SELECT AccountId,
  DATEDIFF('day', MAX(CloseDate), CURRENT_DATE()) AS recency_days,
  COUNT(*) AS frequency,
  AVG(Amount) AS monetary_avg, SUM(Amount) AS monetary_total
FROM won GROUP BY AccountId;
```

(2) Windowed engagement aggregates (point-in-time via a snapshot spine):

```sql
SELECT s.AccountId, s.snapshot_ts,
  COUNT_IF(t.ActivityDate > DATEADD('day',-90,s.snapshot_ts)) AS acts_90d,
  COUNT_IF(t.ActivityDate > DATEADD('day',-30,s.snapshot_ts)) AS acts_30d,
  COUNT(DISTINCT t.WhoId) AS distinct_contacts
FROM feature.snapshot_spine s
LEFT JOIN silver.task t
  ON t.AccountId = s.AccountId AND t.ActivityDate <= s.snapshot_ts  -- no future info
GROUP BY s.AccountId, s.snapshot_ts;
```

(3) Time-in-stage from OpportunityFieldHistory:

```sql
SELECT OpportunityId, NewValue::string AS stage, CreatedDate AS entered_stage_at,
  LEAD(CreatedDate) OVER (PARTITION BY OpportunityId ORDER BY CreatedDate) AS left_st
  DATEDIFF('day', CreatedDate,
     LEAD(CreatedDate) OVER (PARTITION BY OpportunityId ORDER BY CreatedDate)) AS day
FROM silver.opportunityfieldhistory WHERE Field = 'StageName';
```

(4) Recursive account-hierarchy rollup:

```sql
WITH RECURSIVE tree AS (
  SELECT Id, ParentId, Id AS root_id, 0 AS depth FROM silver.account WHERE ParentId I
  UNION ALL
  SELECT a.Id, a.ParentId, t.root_id, t.depth + 1
  FROM silver.account a JOIN tree t ON a.ParentId = t.Id
)
SELECT root_id, COUNT(*) AS hierarchy_size, MAX(depth) AS hierarchy_depth
FROM tree GROUP BY root_id;
```

(5) Point-in-time-correct training-set assembly with ASOF JOIN:

```sql
```

```
WITH spine AS (
  SELECT Id AS LeadId, ConvertedAccountId AS AccountId, CreatedDate AS ts FROM silver
)
SELECT s.LeadId, s.ts, f.recency_days, f.frequency, f.monetary_avg
FROM spine s
ASOF JOIN gold.account_rfm_history f
  MATCH_CONDITION (s.ts >= f.valid_from)      -- closest state at/just before ts
  ON s.AccountId = f.AccountId;
```

(6) Snowpark Python feature pipeline (sketch):

python

```
from snowflake.snowpark import Session, functions as F, Window

def account_engagement(session: Session):
    t = session.table("SILVER.TASK")
    w = Window.partition_by("ACCOUNTID").order_by("ACTIVITYDATE")
    return (t.with_column("acts_cum", F.count("*").over(w))
             .group_by("ACCOUNTID")
             .agg(F.count("*").alias("acts_total"),
                  F.count_distinct("WHOID").alias("distinct_contacts")))
```

3.5 Snowflake Feature Store & Model Registry (verified)

- **Feature Store** materialises features as **Feature Views** and generates training data with point-in-time correctness. `FeatureView(name, entities, feature_df, timestamp_col=None, refresh_freq=None, ...)` supports a `timestamp_col` ("name of the timestamp column for point-in-time lookup when consuming the feature values") and `refresh_freq` (e.g., `"5 minutes"`, a cron expression, or `DOWNSTREAM`; minimum 1 minute).
- Training data via **`generate_training_set(spine_df, features, *, save_as=None, spine_timestamp_col=None, spine_label_cols=None, ...)`**. The **spine** is "a DataFrame containing the entity IDs in source data, the time stamp, label columns, and additional columns." `spine_timestamp_col` is "Name of timestamp column in spine_df that will be used to join time-series features. If spine_timestamp_col is not none, the input features also must have timestamp_col." Features are retrieved "with point-in-time correctness with respect to the provided time stamp" — the store does the ASOF join for you. `generate_dataset(...)` returns a versioned, immutable **Dataset** (Parquet) for reproducibility; `generate_training_set` returns an ephemeral Snowpark DataFrame unless `save_as` materialises a table.
- **Model Registry** stores models as schema-level objects with versions/metrics/metadata, serves inference via Python/SQL/REST, supports RBAC (USAGE/READ), integrates with ML Lineage and ML Observability. Models trained by SQL ML Functions (e.g., FORECAST) do **not** appear in the registry.

- **time travel & zero-copy cloning**: use time travel to reproduce a training snapshot; zero-copy `CLONE` for isolated dev copies of feature/gold schemas without storage duplication.

---

4. Model Selection — The Other Top Emphasis

4.1 Probabilistic BTYD models (benchmark & feature source)

- **Pareto/NBD** (Schmittlein, Morrison & Colombo 1987, *Management Science*) — original "counting your customers"; hard to estimate.
- **BG/NBD** (Fader, Hardie & Lee 2005, "Counting Your Customers the Easy Way: An Alternative to the Pareto/NBD Model," *Marketing Science* 24(2):275–284): "We develop a new model, the beta-geometric/NBD (BG/NBD), which represents a slight variation in the behavioral story associated with the Pareto/NBD, but it is vastly easier to implement… its parameters can be obtained quite easily in Microsoft Excel." Recency & frequency are sufficient statistics. **MBG/NBD** assumes non-repeat customers can still be alive.
- **BG/BB** — discrete/contractual analogue (renewal-epoch data). **Gamma-Gamma** spend model for E[monetary|transaction], relying on frequency–monetary independence. **Covariate/heterogeneity extensions**: Pareto/NBD with covariates, hierarchical Bayes, Pareto/GGG.
- **Implementations**: `lifetimes` (Python) is in **maintenance mode** — its README/PyPI now states "A project has emerged as a successor to lifetimes, PyMC-Lab/PyMC-Marketing, please check it out!" **PyMC-Marketing's CLV module** provides "BG/NBD model for continuous time, non-contractual modeling; Pareto/NBD… with covariates; Shifted BG model for discrete time, contractual modeling with cohorts and covariates; BG/BB model," plus Gamma-Gamma — the recommended Python route; **CLVTools** and **BTYD/BTYDplus** in R.
- **Why they break in B2B**: small N, hard to include covariates, non-stationarity, channel censoring. **Use as benchmark + feature source** (P(alive)/expected-transactions), not the primary engine.

4.2 Survival / time-to-event models

Given heavy right-censoring (accounts still alive; renewals pending), survival framing suits **renewal/churn timing** and **time-to-refresh**: Cox PH; AFT; discrete-time survival (logistic on person-period); Random Survival Forests; **XGBoost** `survival:cox` (predictions on hazard-ratio scale) and `survival:aft` (`aft_loss_distribution` ∈ {normal, logistic, extreme}; supports censored/interval labels via `label_lower_bound`/`label_upper_bound`); DeepSurv; DeepHit. Use **competing risks** when churn vs upgrade vs downgrade are distinct exits. Evaluate with the concordance index. `XGBoost`

4.3 Supervised ML regression / two-stage (the production default)

GBDTs (**XGBoost, LightGBM, CatBoost**) are the empirically established best default for tabular CRM data:

- Grinsztajn, Oyallon & Varoquaux (2022, NeurIPS Datasets & Benchmarks track; arXiv:2207.08815), *"Why do tree-based models still outperform deep learning on typical*

tabular data?" — across a standard set of 45 datasets, "tree-based models remain state-of-the-art on medium-sized data (~10K samples) even without accounting for their superior speed." The three NN-design challenges they identify are to "1. be robust to uninformative features, 2. preserve the orientation of the data, and 3. be able to easily learn irregular functions" — exactly the properties GBDTs already have and that make them well-suited to noisy CRM tables.

- Shwartz-Ziv & Armon (2022, *Information Fusion* 81:84–90), *"Tabular Data: Deep Learning is Not All You Need"* — XGBoost generally beat bespoke deep models across datasets; ensembling helped.
- **CatBoost** (Prokhorenkova et al. 2018, NeurIPS) — ordered target statistics + ordered boosting remove the leakage/prediction-shift of naïve encoding, an advantage for high-cardinality CRM categoricals (industry, source, campaign, product family, owner). arXiv
- Tabular **foundation models** (TabPFN, TabPFN v2/v2.5 in 2024–2025) advance quickly for small tabular tasks, but their fit for right-censored, leakage-sensitive, cost-weighted CLV at B2B scale is unproven — a **research challenger**, not production.

4.4 Zero-inflation and heavy-tailed targets (central to CLV)

- **Two-stage / hurdle**: classifier for P(convert) × regressor for E[value|convert]. Interpretable; optimise each stage's metric; matches the decision decomposition.
- **Tweedie** (compound Poisson-Gamma): one model for zero-inflated positive-continuous targets. XGBoost/LightGBM `objective='reg:tweedie'`/`'tweedie'` with `tweedie_variance_power` $p \in (1,2)$ — $p \to 1$ Poisson-like, $p \to 2$ Gamma-like; tune p on validation (CLV often $p \approx 1.1$–1.5).
- **Gamma/log-normal GLMs**; `log1p` transforms with awareness of the **retransformation (Jensen's-inequality) bias** — $E[\exp(x)] \neq \exp(E[x])$; apply a smearing/analytic correction when back-transforming.
- **Zero-Inflated Lognormal (ZILN)** — Wang, Liu & Miao (2019), *"A Deep Probabilistic Model for Customer Lifetime Value Prediction"* (arXiv:1912.07753; Google's open-source `google/lifetime_value`). Models LTV as "a mixture of zero-point mass and a lognormal distribution," trained by minimising the sum of a BCE (churn/convert) loss and a lognormal loss — a principled joint hurdle model yielding a **predictive distribution** (uncertainty). The authors "recommend the normalized Gini coefficient to quantify model discrimination and decile charts to assess model calibration"; in their experiments the ZILN loss achieved normalized Gini 0.190 vs 0.184 (MSE) and smaller decile-level MAPE (0.176 vs 0.210), and the best ZILN DNN reached total profit \$15,498.24 — a 5% relative gain over the competition winner's \$14,712.24. ZILN fits CLV exactly (zero spike + heavy right tail); implement its loss on a GBDT/NN or approximate with the two-stage GBDT.

4.5 Deep learning & sequence models

RNN/LSTM/TCN/Transformers on event sequences with entity embeddings beat GBDTs **only** with large data and rich sequences. At typical B2B volumes (thousands of accounts) they usually **underperform** well-tuned GBDTs and are harder to keep leakage-free. Revisit only with very large, granular event streams (e.g., telemetry).

4.6 Quantile / distributional & uncertainty-aware

CLV decisions need **uncertainty, not just a point estimate** (pCLV $50k ± $5k vs ± $60k are different decisions). Use quantile regression (GBDT `objective='quantile'`), **NGBoost** (predictive distributions), or **conformal prediction** (distribution-free intervals). ZILN gives a distribution natively.

4.7 Causal / uplift considerations

Predicting CLV ≠ predicting the **incremental effect** of an action on CLV. Prioritising by pCLV alone over-serves accounts that would convert **anyway** ("sure things") and wastes effort. Use **uplift modelling** for spend allocation: two-model/T-learner, class-transformation, causal forests, and meta-learners **S/T/X/R-learner** and **DR-learner** (Künzel et al. 2019, PNAS 116(10):4156–4165; Nie & Wager 2021) via **EconML** / **CausalML** / **DoWhy**. Honest framing: **rank leads for the sales queue by pCLV; allocate incremental marketing budget by uplift.** arxiv GitHub

4.8 Recommended architecture & decision framework

Recommended production system:

1. **Stage A — conversion**: LightGBM/XGBoost/CatBoost classifier → calibrated P(convert) within horizon.
2. **Stage B — value | convert**: GBDT with **Tweedie** objective (or a ZILN head) → E[value|convert]; retransformation-corrected if log-space.
3. **Value of a lead** = P(convert) × E[value|convert], plus a **prediction interval** (quantile/conformal).
4. **Survival component** for install-base accounts → renewal probability & months-to-refresh, folded in as expansion value.
5. **BTYD/Gamma-Gamma benchmark** (PyMC-Marketing) in parallel; its P(alive)/expected-transactions feed Stages A/B as features.
6. **Calibration** (Platt/isotonic) on Stage A so probabilities are decision-grade.

Baseline hierarchy (always measure lift over the simplest): heuristic RFM segments → logistic/Tweedie GLM → GBDT two-stage → advanced (ZILN/survival/uplift). Don't ship a complex model that fails to beat the RFM/GLM baseline out of sample.

Decision table — data situation → model:

Situation	Recommended
Pure transaction log, many customers, no covariates	BG/NBD + Gamma-Gamma
Rich CRM covariates, medium N, zero-inflated $ target	**Two-stage GBDT (classifier × Tweedie)** ✅ default
Need full predictive distribution / uncertainty	ZILN or NGBoost/quantile + conformal
Renewal/refresh timing, heavy censoring	Survival (Cox/AFT, XGBoost survival, RSF)
Allocating incremental spend	Uplift / meta-learners (EconML/CausalML)
Very large, granular event sequences	Sequence DL (challenger)

4.9 Evaluation & validation

- **Temporal / out-of-time validation only** — never random K-fold on time-series CRM data (it leaks the future). Train on cohorts up to `T`, validate on `T→T+H`; use rolling-origin backtests.
- **Conversion stage**: AUC/PR-AUC (PR-AUC for imbalance) + **calibration** (reliability curves, Brier).
- **Value/CLV**: **normalised Gini** + **decile/lift charts** + **calibration** (the Google ZILN combination) as the primary lens; Spearman rank correlation; **top-decile capture rate**. MAE/RMSE/MAPE are **weak on heavy-tailed targets** (dominated by whales / undefined near zero) — report but don't optimise them alone.
- **Survival**: concordance index (C-index), time-dependent AUC.
- **Business-level**: profit curves, expected profit per decision, and **CAC:LTV ratio validation** by segment/channel — the metrics leadership acts on.
- **Right-censoring in labels**: for a fixed horizon H, only use leads with ⩾H of observable follow-up (or explicitly censor and use survival). Choose H to match the decision (12-mo for lead routing; 36-mo for ABM/strategic).

---

5. MLOps & Production Architecture on Snowflake

5.1 Reference architecture (text)

Salesforce + HubSpot → **ELT** (Fivetran/Airbyte/native + history/property-history capture) → Snowflake **RAW/bronze** → **silver** (typed, deduped, SCD2, identity-resolved) via **dbt or Dynamic Tables** → **gold** (account/lead marts) → **Snowflake Feature Store** (Feature Views, point-in-time training sets) → **training** (Snowpark ML / ML Jobs / Container Runtime) → **Model Registry** (versions, metrics) → **batch scoring** (registry inference / vectorised Python UDF) → **Reverse ETL** write-back (Hightouch/Census, or Salesforce Bulk API 2.0 / HubSpot properties API) → activation in Salesforce/HubSpot.

5.2 Orchestration

- **Dynamic Tables** for declarative silver/gold pipelines (define target lag; Snowflake handles

incremental refresh & dependency ordering) — per Snowflake's decision guide, "the recommended approach for new multi-table SQL pipelines… especially pipelines with joins, aggregations, or window functions." Snowflake Documentation

- **Streams + Tasks** when you need MERGE/upsert with compound keys, stored procedures, or explicit CRON/DAG control (Dynamic Tables don't support MERGE). Streams capture row-level CDC between two transactional points; Task graphs chain steps as DAGs. Snowflake Documentation Snowflake Documentation
- **dbt** (Cloud/Core) for transformation modelling/testing; **Airflow/Dagster/Prefect** if already in use.
- **Snowpark Container Services / ML Jobs** for heavier training or external-IDE dispatch; external SageMaker/Vertex/Databricks only with existing investment.

5.3 Snowflake ML specifics (verified; check GA/preview in your account)

- **Snowpark ML Modeling API** (`snowflake-ml-python`) — sklearn-style preprocessing/estimators running in Snowflake.
- **Feature Store** — Feature Views + `generate_training_set`/`generate_dataset` (Section 3.5).
- **Model Registry** — versioned model objects; Python/SQL/REST inference; RBAC.
- **ML Jobs / Container Runtime / Notebooks** — pipeline automation and interactive dev.
- **Cortex ML Functions** — SQL-callable **FORECAST**, **ANOMALY_DETECTION** (both GA), **CLASSIFICATION**, plus **TOP_INSIGHTS**/Contribution Explorer. Realistic fit: **CLASSIFICATION** can serve a quick conversion-propensity baseline in pure SQL; **FORECAST** suits aggregate demand/refresh forecasting; **ANOMALY_DETECTION** for monitoring pipeline volumes. They are **not** a substitute for the custom two-stage/Tweedie/ZILN model (no Tweedie/hurdle/survival objective, limited control) — use for baselines and monitoring. **Verify GA vs preview and exact behaviour in your account.**
- **UDFs/UDTFs & vectorised Python UDFs** for scalable scoring; **Snowflake Notebooks** for development.

5.4 Monitoring (verified)

- **Snowflake ML Observability (Model Monitor)** — "track the quality of production models you have deployed via the Snowflake Model Registry across multiple dimensions, such as performance, drift, and volume," including per-segment monitoring. Created via `CREATE MODEL MONITOR … WITH MODEL=… VERSION=… FUNCTION=… SOURCE=… WAREHOUSE=… REFRESH_INTERVAL=… AGGREGATION_WINDOW=… TIMESTAMP_COLUMN=… [BASELINE=…]`. Metrics via table functions `MODEL_MONITOR_DRIFT_METRIC` (supports **POPULATION_STABILITY_INDEX (PSI)**, JENSEN_SHANNON, DIFFERENCE_OF_MEANS, WASSERSTEIN — a **baseline** table is required for drift), `MODEL_MONITOR_PERFORMANCE_METRIC`, `MODEL_MONITOR_STAT_METRIC`. **Constraints**: supports **regression and binary classification** only; one monitor per model version; requires `snowflake-ml-python` ⩾ 1.7.1; ⩽500 features, ⩽250 monitors, ⩾1-day aggregation window; timestamps `TIMESTAMP_NTZ`, prediction/actual columns `NUMBER`. GA in all regions per Snowflake's blog; a specific dated GA docs release-note was not located, so **confirm status in your account.** (Do not confuse with Cortex *AI* Observability for GenAI apps, GA 2025-07-31.)

- **What to monitor for B2B CLV**: feature drift (PSI/KS), prediction drift, and — critically — **delayed-label performance decay**: because B2B sales cycles are long, true labels arrive months later, so track leading indicators (stage-progression rates, calibration on the subset with matured labels) and run **champion/challenger + shadow** deployments. **Retraining triggers**: PSI above threshold on key features, calibration drift, or a scheduled cadence (e.g., quarterly), whichever first.

5.5 Governance

Model cards/documentation in the registry; reproducibility via Dataset versioning + time travel + zero-copy clone; **feature lineage** via ML Lineage; **PII/GDPR/CCPA** for scoring individuals — apply Snowflake **dynamic data masking** and **row-access policies**, minimise PII in features, and log purpose. **Fairness/compliance**: firmographic proxies (geography, industry, size) can encode bias and, for individual-level scoring, may carry regulatory exposure — document rationale, keep humans in the loop for high-stakes routing, avoid protected-class proxies.

5.6 Activation

- **Salesforce**: write scores to **custom fields** on Lead/Account (e.g., `pCLV__c`, `pConvert__c`, `CLV_Decile__c`, `CLV_Model_Version__c`) via Bulk API 2.0 or Reverse ETL; drive lead routing/assignment rules and SLA timers off the decile.
- **HubSpot**: write to **custom contact/company/deal properties** via the properties API; enrol in workflows and list segmentation by score bucket.
- **Prove value**: deploy behind a **randomised holdout** (a fraction of leads routed by the old method) and measure incremental conversion/pipeline/revenue — an A/B test of the *model as an intervention*, not just offline AUC.

Recommendations (staged, with benchmarks that change the plan)

Phase 1 — Crawl (≈4–8 weeks; 1 data engineer + 1 data scientist). Land Salesforce + HubSpot in Snowflake **with history and property-history**; build silver SCD2 tables + domain-based identity resolution; ship an **RFM-segment + logistic/Tweedie GLM** baseline; write one score back to a `pCLV__c` custom field; stand up a temporal (out-of-time) backtest harness.

- *Benchmark to advance:* the GLM/RFM baseline is calibrated and beats random routing on top-decile capture in an out-of-time test. If you cannot beat it, fix data/leakage before adding model complexity.

Phase 2 — Walk (≈2–3 months; add partial MLOps). Stand up the **Snowflake Feature Store** with point-in-time training sets (`generate_training_set(spine_timestamp_col=...)`); build the **two-stage GBDT** (calibrated classifier × Tweedie regressor); run SHAP-based + stability feature selection inside the CV loop; register in the **Model Registry**; batch-score and activate via Reverse ETL; add a **Model Monitor** with PSI drift.

- *Benchmark to advance:* two-stage GBDT beats the GLM on normalised Gini AND stays calibrated on a temporal holdout; if not, prefer the simpler model.

Phase 3 — Run (ongoing). Add a **survival** component (refresh/renewal timing from Asset

(`UsageEndDate`/`InstallDate`), **ZILN/uncertainty** outputs, and **uplift models** (EconML/CausalML) for marketing-spend allocation; institute champion/challenger + shadow deployments; automate retraining on PSI/calibration triggers; prove business value with a randomised routing holdout.

- *Thresholds that change the plan:* firmographic-only cold-start sub-model if lead behaviour is sparse; switch spend decisions from pCLV to uplift once you can run controlled treatment/holdout data; consider sequence DL only if you accumulate tens of thousands of accounts with rich event streams.

Data-readiness checklist: history objects enabled/ingested; field-history tracking on needed fields (⩽20/object limit); identity resolution validated; margin data availability confirmed; channel/end-customer resolution built; label maturity (⩾ horizon H of history); documented leaky-field quarantine list.

Minimum viable data volumes (rules of thumb — validate empirically): BTYD needs a reasonable transacting population (unstable with very few repeat buyers); the two-stage GBDT is workable from a few thousand labelled leads with enough positives (watch class imbalance); deep/sequence models realistically need tens of thousands+ with rich sequences.

Known B2B pitfalls & mitigations:

- **Survivorship bias** — include lost/dormant accounts and censored leads, not only current customers.
- **Sales-rep selection bias / self-fulfilling prophecy** — reps work high-value leads, so labels reflect rep behaviour and the model re-learns rep priors. Mitigate with uplift framing, exploration holdouts (route some low-scored leads anyway), and feedback-loop monitoring.
- **Simpson's paradox** — evaluate within key segments (SMB vs enterprise, direct vs channel), not just pooled.
- **Small-sample instability** — stability selection, regularisation, hierarchical pooling.
- **Channel/distributor data gaps** — explicit end-customer resolution + an "unknown end customer" flag.
- **Cold start** — firmographic-only sub-model for brand-new leads, blending toward behavioural models as engagement accrues.

Caveats

- **Field/object names**: standard Salesforce and HubSpot API names above are from official references, but **custom fields (`__c`, custom HubSpot properties) and which history/analytics fields are populated are org- and edition-specific** — verify every field in your instance. HubSpot predictive scoring (`hs_predictivecontactscore`) and recurring-revenue deal metrics (`hs_arr`/`hs_mrr`) are **edition-gated** (Marketing/Sales Hub tiers); Salesforce Einstein scoring is separately licensed. Confirm the exact predictive-score internal name via HubSpot's properties API (historic `..._v2` naming has appeared).
- **Snowflake feature status**: Feature Store, Model Registry, ML Observability, Dynamic Tables, ASOF JOIN, and Cortex ML Functions are cited from Snowflake docs, but **GA vs preview and exact API signatures change** — confirm in your account and pin `snowflake-`

`ml-python`) versions. The dated GA release-note for ML Observability was confirmed via Snowflake's corporate blog rather than a docs release-note.

- **BTYD-vs-ML accuracy**: the literature is **mixed** — BTYD can match or beat ML when only a clean transaction log exists, while covariate-rich ML wins with strong features. Treat the recommendation (two-stage GBDT primary, BTYD benchmark) as evidence-based default, not universal law; measure on your data.
- **Numbers**: effort estimates and volume thresholds are rules of thumb, not guarantees; validate against your data and team.